



**RAJALAKSHMI  
ENGINEERING COLLEGE**

An AUTONOMOUS Institution  
Affiliated to ANNA UNIVERSITY, Chennai

**DEPARTMENT OF COMPUTER SCIENCE AND  
ENGINEERING**

**CS23432 – Software Construction**

**(REGULATION 2023)**

**RAJALAKSHMI ENGINEERING COLLEGE  
Thandalam, Chennai-602015**

Name: Prajeet V

Register No: 240701389

Year / Branch / Section: II / CSE

Semester: IV

Academic Year: 2025 - 2026

# INDEX

| S.No. | Date       | Title                                                                           |
|-------|------------|---------------------------------------------------------------------------------|
| 1.    | 07-02-2026 | Azure DevOps Environment Setup                                                  |
| 2.    | 14-02-2026 | Azure Project Setup & User Stories                                              |
| 3.    | 21-02-2026 | Epics, Features, User Stories Creation                                          |
| 4.    | 28-02-2026 | Sprint Planning using Azure Boards                                              |
| 5.    | 07-03-2026 | Poker Estimation using Azure Boards.                                            |
| 6.    | 14-03-2026 | Designing Class and Sequence Diagrams for Product Category Management System.   |
| 7.    | 04-04-2026 | Designing Architectural and ER Diagrams for Product Category Management System. |
| 8.    | 11-04-2026 | Testing – Test Suites and Test Cases in Azure.                                  |
| 9.    | 18-04-2026 | Load Testing and CI/CD Pipeline Setup.                                          |
| 10.   | 25-04-2026 | GitHub: Project Structure & Naming Conventions.                                 |

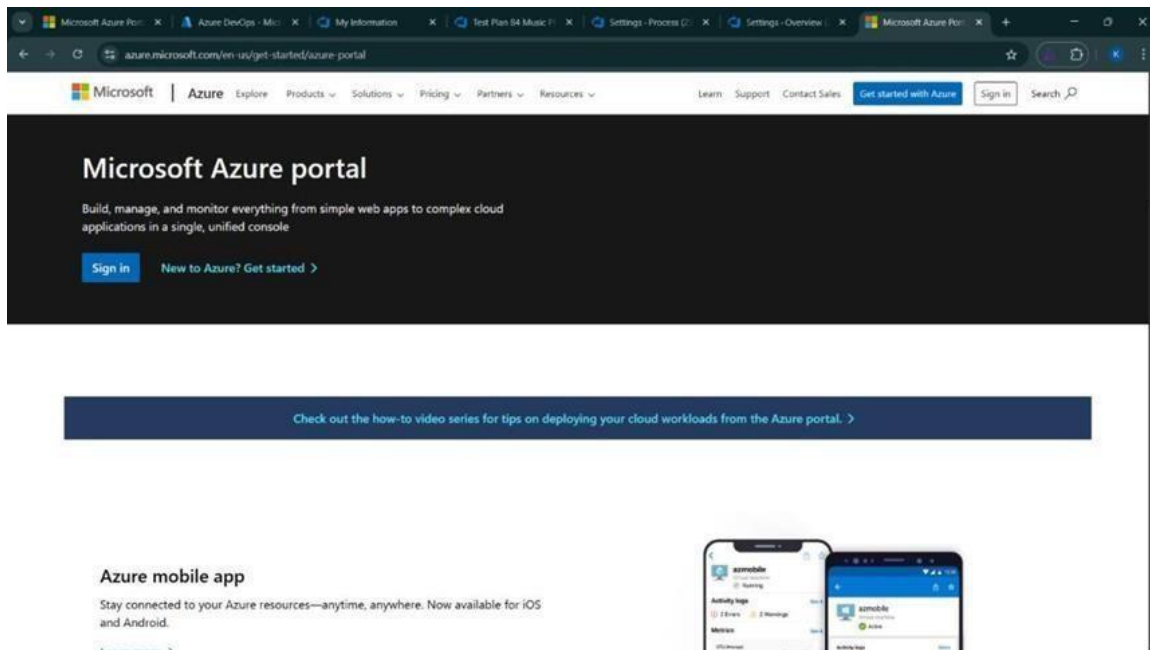
|            |                                       |
|------------|---------------------------------------|
| EXP NO: 01 | <b>AZURE DEVOPS ENVIRONMENT SETUP</b> |
|------------|---------------------------------------|

### Aim:

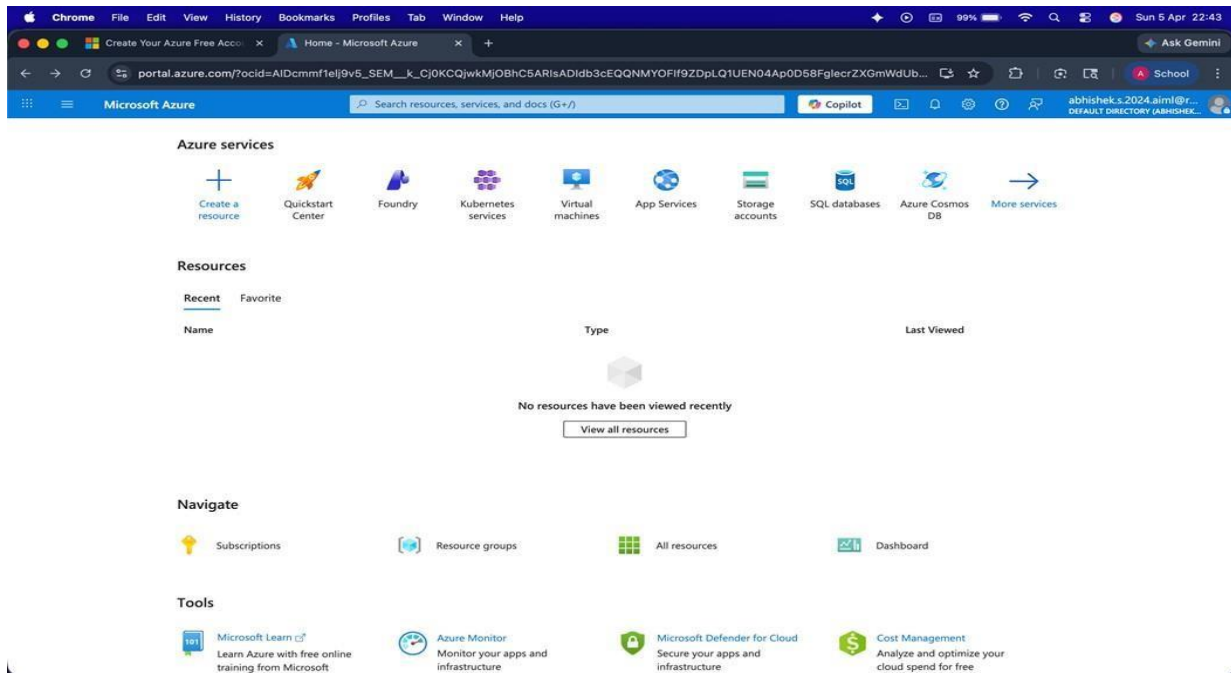
To set up and access the Azure Devops environment by creating an account and workspace for project management.

### INSTALLATION

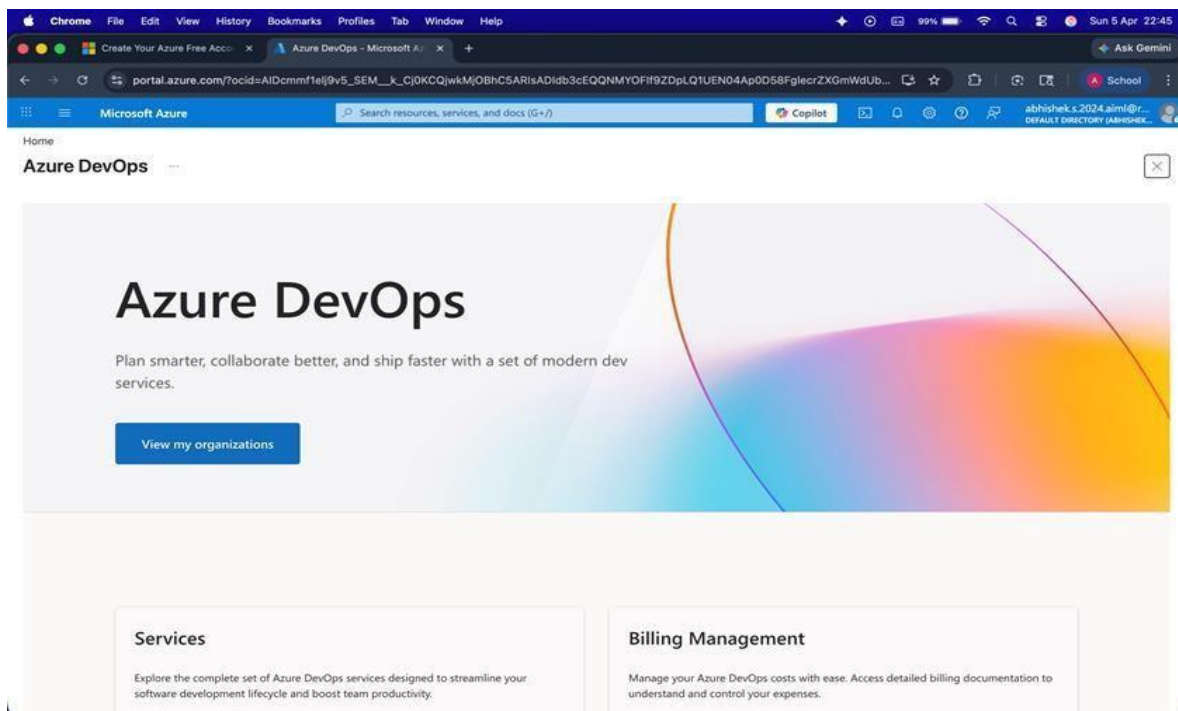
1. Open your web browser and go to: <https://dev.azure.com>  
Sign in using your Microsoft account credentials.  
If you do not have an account, create one using Sign Up option.



## 2. Azure DevOps home page



## 3. Open DevOps environment in the Azure platform by typing **Azure DevOps Organizations** in the search bar.



4. Click on **New Project**, create a new organization/project, and access the Azure DevOps dashboard successfully..

The screenshot shows the Azure DevOps interface for a project named 'hospital-appointment-system'. The left sidebar contains navigation links: Overview, Summary, Dashboards, Wiki, Boards, Pipelines, and Artifacts. The main content area is divided into two columns. The left column, titled 'About this project', contains a project description, objective, key features, and a list of items. The right column, titled 'Project stats', shows metrics for Boards, Repos, and Pipelines, along with a list of members.

**Project Description:** Hospital Appointment System  
The Hospital Appointment System is a web-based application designed to simplify and automate the process of booking and managing patient appointments. This system enables users to schedule appointments with doctors efficiently while ensuring scalability, reliability, and performance under real-world traffic conditions.

**Objective**  
The primary goal of this project is to:

- Provide a simple and efficient API for booking hospital appointments
- Handle multiple concurrent users using scalable backend architecture
- Evaluate system performance using modern load testing tools
- Demonstrate cloud deployment and DevOps integration

**Key Features**

- Appointment Booking API**  
Users can book appointments by providing:

- Name
- Doctor
- Date

**High Performance Backend**  
Built using Nodejs and Express for fast request handling.

**Database Integration**  
Uses MySQL to store appointment data securely.

**Cloud Deployment**  
Backend is deployed on Render, making it publicly accessible.

**Project stats** (Period: Last 7 days)

**Boards**

- 0 Work items created
- 0 Work items completed

**Repos**

- 0 Pull requests opened
- 0 Commits by 0 authors

**Pipelines**

- 0% Builds succeeded

**Members** (5)

- SS, PV, GS, PS

## Result:

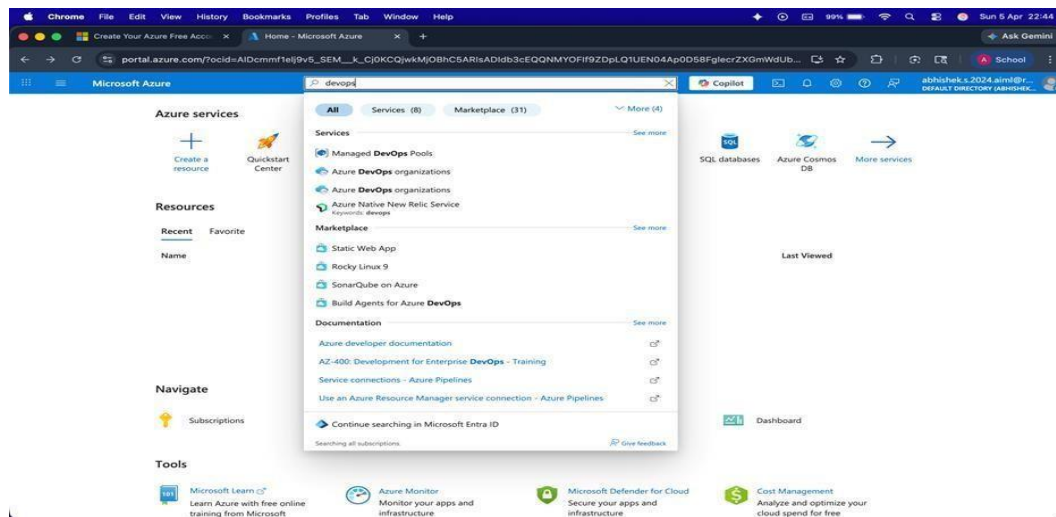
Successfully accessed the Azure environment and created a new workspace/site for project management.

|            |                                                                                    |
|------------|------------------------------------------------------------------------------------|
| EXP NO: 02 | <h1 style="text-align: center;">AZURE PROJECT SETUP AND USER STORY MANAGEMENT</h1> |
|------------|------------------------------------------------------------------------------------|

## Aim:

To set up an Azure DevOps project for efficient collaboration and agile work management.

- Create an Azure DevOps Account
- Create the First Project in Your Workspace
  - a. After setup, create your first project.
  - b. Click **New Project**
  - c. Enter details:  
 Name: Product Category Management System  
 Description: Category handling system  
 Template: Scrum  
 Access: Private/Open
  - d. Click Create
- Open project dashboard
- Project dashboard
- To manage user stories:
  - a. Go to **Boards** → **Work Items**
  - b. Click **New Work Item** → **User Story**
  - c. Enter details



**Result:** Successfully created an Azure DevOps project with user story management

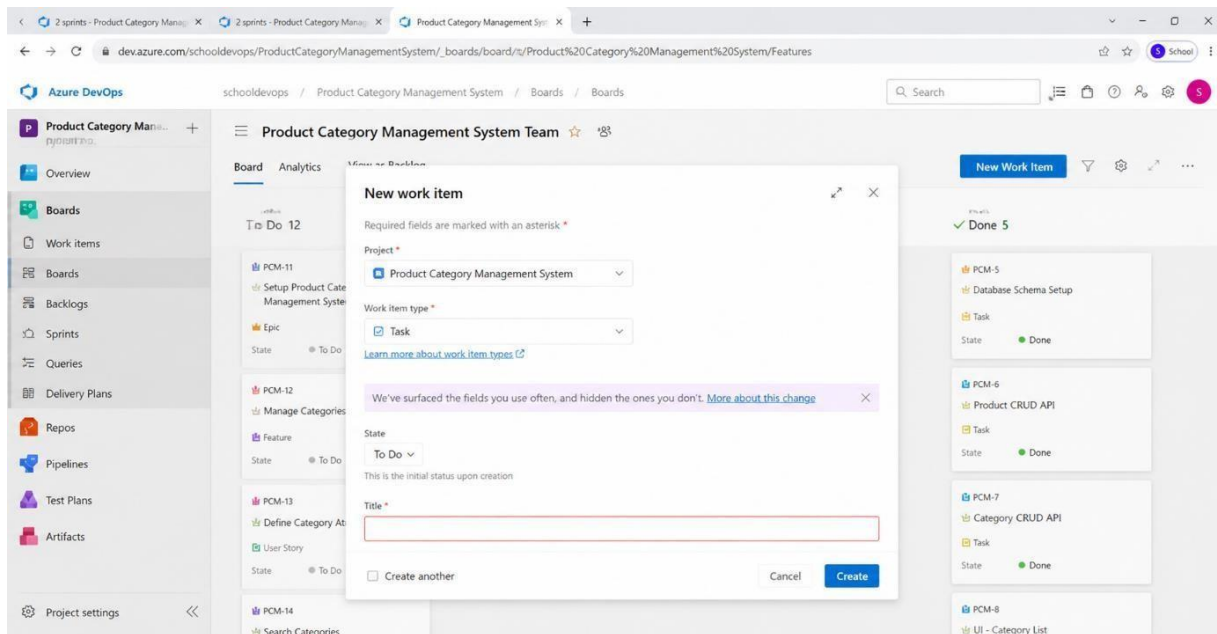
EXP NO: 03

# SETTING UP EPICS, TASKS, AND USER STORIES FOR PROJECT PLANNING

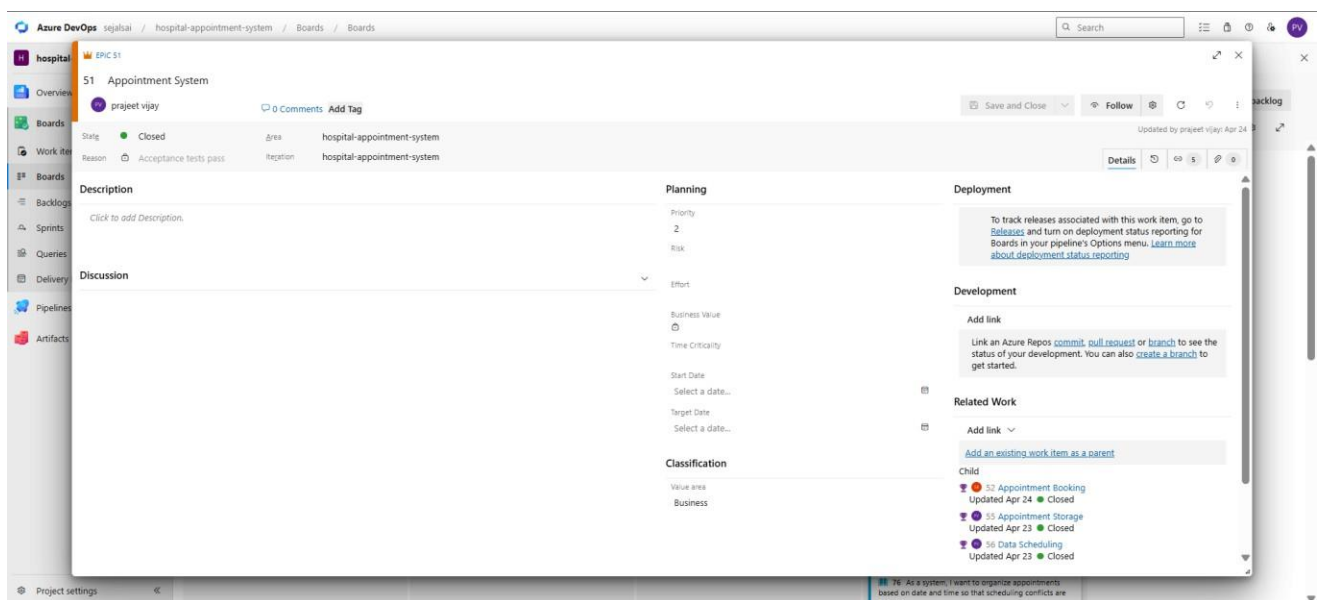
## Aim:

To learn about how to create epics, tasks, user stories, and backlogs for your assigned project.

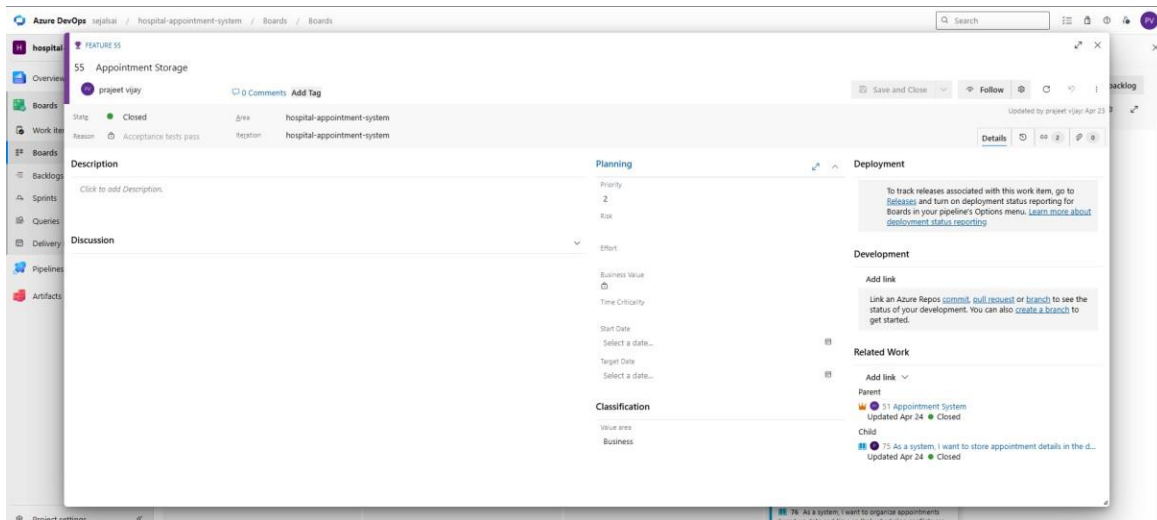
## Create Epic, User Stories, Task



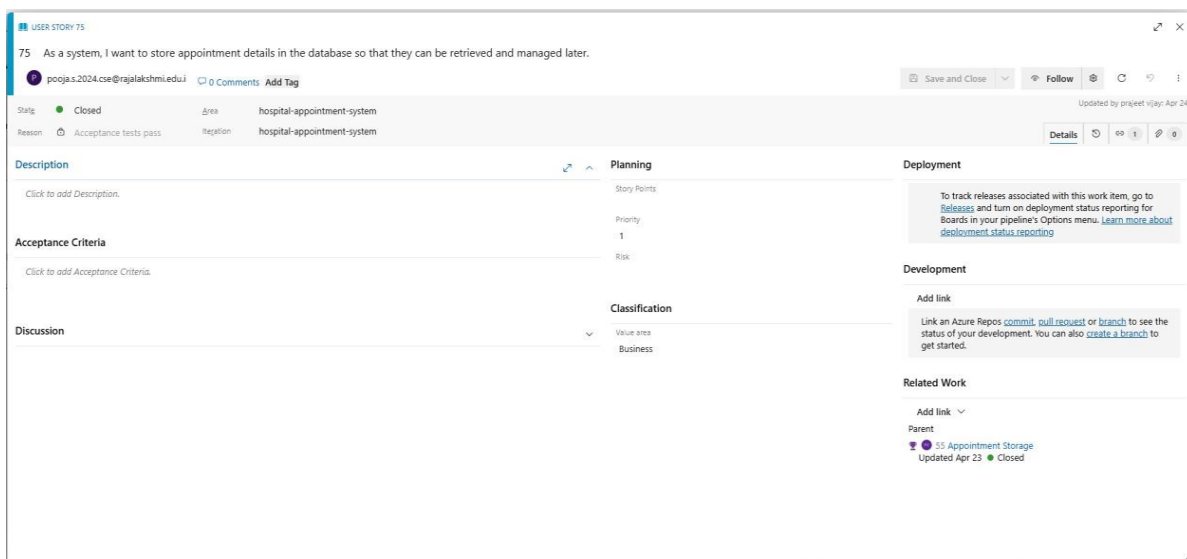
## 1.Fill in Epics



## 2.Fill in Features



## 3.Fill in User Story Details



## Result:

Thus, the creation of epics, features, user story and task has been created success



**EXP NO: 04**

# **SPRINT PLANNING IN AZURE**

## **Aim:**

To assign user story to specific sprint for the Hospital Appointment Management System

## **Sprint Planning**

1. Open **Boards** → **Sprints**
2. Create:
  - Sprint 1
  - Sprint 2
3. Drag user stories into sprint

## **Sprint 1**

The screenshot displays the Azure DevOps interface for a project named 'hospital-appointment-system'. The left sidebar shows the navigation menu with 'Sprints' selected. The main area shows a Kanban board for the 'hospital-appointment-system Team'. The board is titled 'Sprint 1 - Core Setup - Basic user access simple data display' and is set for 'March 20 - March 25' (4 work days). The board is divided into columns: 'New', 'Active', 'Resolved', and 'Closed'. A 'Collapse all' button is visible above the 'New' column. The 'Closed' column contains six work items, each with a title, status, and assignee. The work items are: '18 Create appointments table in database' (Closed, prajet vijay), '25 Create login page UI using HTML' (Closed, Pragith S), '26 Create registration page UI using HTML' (Closed, sharan bose), '27 Create login page UI using HTML' (Closed, sharan bose), '31 Design input forms for user data' (Closed, prajet vijay), and '34 Connect frontend to login API' (Closed, prajet vijay).

| Work Item ID | Title                                  | Status | Assignee     |
|--------------|----------------------------------------|--------|--------------|
| 18           | Create appointments table in database  | Closed | prajet vijay |
| 25           | Create login page UI using HTML        | Closed | Pragith S    |
| 26           | Create registration page UI using HTML | Closed | sharan bose  |
| 27           | Create login page UI using HTML        | Closed | sharan bose  |
| 31           | Design input forms for user data       | Closed | prajet vijay |
| 34           | Connect frontend to login API          | Closed | prajet vijay |

## Sprint 2

Azure DevOps [sejalrai](#) / [hospital-appointment-system](#) / [Boards](#) / [Sprints](#)

hospital-appointment-... +

Overview

Boards

Work items

Boards

Backlogs

Sprints

Queries

Delivery Plans

Pipelines

Artifacts

Project settings

hospital-appointment-system Team

Taskboard Backlog Analytics

Iteration 2 Person: All

No iteration dates

Collapse all

New Active Resolved Closed

7 User Login and Authentication  
Closed  
Unassigned

8 Install and configure JWT package  
Closed  
Unassigned

10 Implement password verification using bcrypt  
Closed  
Unassigned

11 Protect API routes using authentication middleware  
Closed  
Unassigned

12 Test login and protected routes using API tester  
Closed  
Unassigned

13 Update login API to generate JWT token  
Closed  
Unassigned

15 Create middleware to

### Result:

The Sprints are created for the Hospital Appointment Management System

|                   |                                     |
|-------------------|-------------------------------------|
| <b>EXP NO: 05</b> | <b>POKER ESTIMATION USING AZURE</b> |
|-------------------|-------------------------------------|

**Aim:**

Create Poker Estimation for the user stories – Hospital Appointment Management System.

**Poker Estimation**

| Points | Meaning                                                             |
|--------|---------------------------------------------------------------------|
| 1      | Simple UI text update or small bug fix                              |
| 2      | Login/Register form validation                                      |
| 3      | Basic database connection and API setup                             |
| 5      | Appointment booking module with frontend and backend                |
| 8      | Dashboard with appointment history and management                   |
| 13     | Integration of Azure DevOps, CI/CD pipeline, and database           |
| 21     | Complete Hospital Management System with all modules and deployment |

**Procedure:**

- Assign points:
  - Add Category → 3
  - Delete Category → 2
  - Search → 1

| Sprint   | Total SP | Duration | Velocity          | Focus                                         |
|----------|----------|----------|-------------------|-----------------------------------------------|
| Sprint 1 | 32 SP    | 2 weeks  | ~13SP/dev/sprint  | Login, Registration, Database Setup, Basic UI |
| Sprint 2 | 48 SP    | 2 weeks  | ~31 SP/dev/sprint | Azure Pipelines, Testing, CI/CD Integration   |

**Result:**

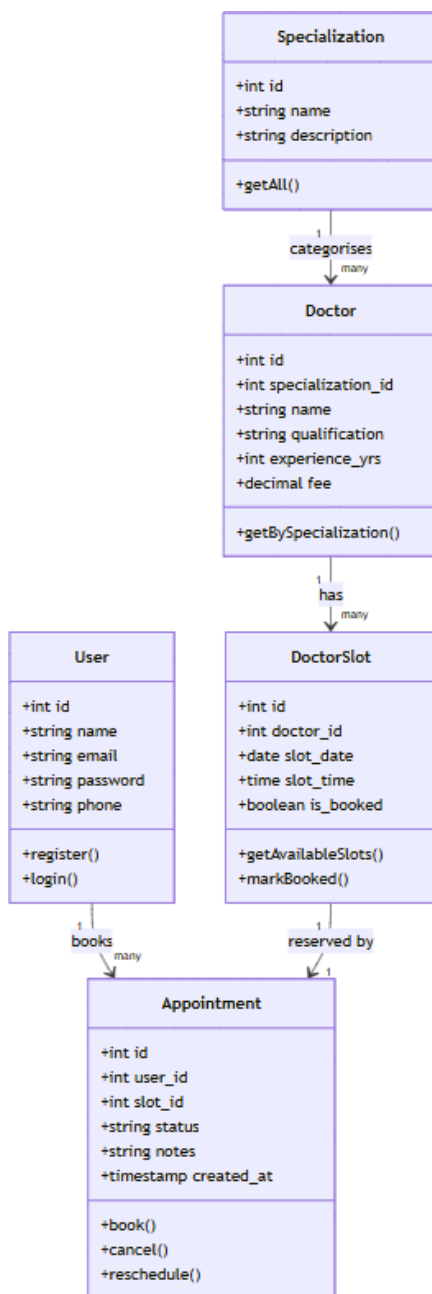
The Estimation/Story Points is created for the project using Poker Estimation.

|            |                                                                                         |
|------------|-----------------------------------------------------------------------------------------|
| EXP NO: 06 | <b>DESIGNING CLASS AND SEQUENCE DIAGRAMS FOR<br/>PRODUCT CATEGORY MANAGEMENT SYSTEM</b> |
|------------|-----------------------------------------------------------------------------------------|

**Aim:**

To Design a Class Diagram and Sequence Diagram for the given Hospital Appointment Management System.

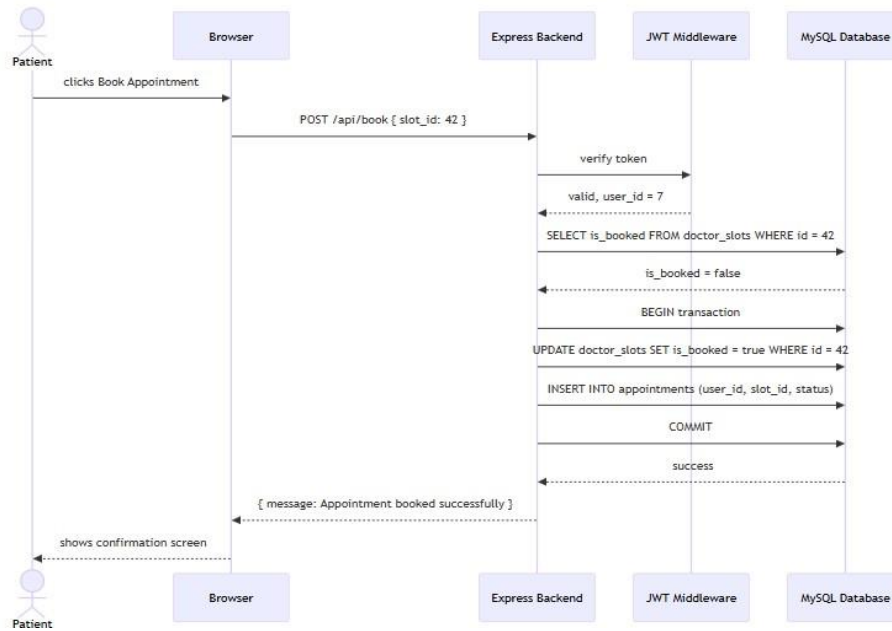
**6A. Class Diagram**



## 6B. Sequence Diagram

Sequence Diagram — Appointment Booking Flow

This diagram shows the step-by-step conversation between the browser, backend, authentication middleware, and database when a patient books an appointment.



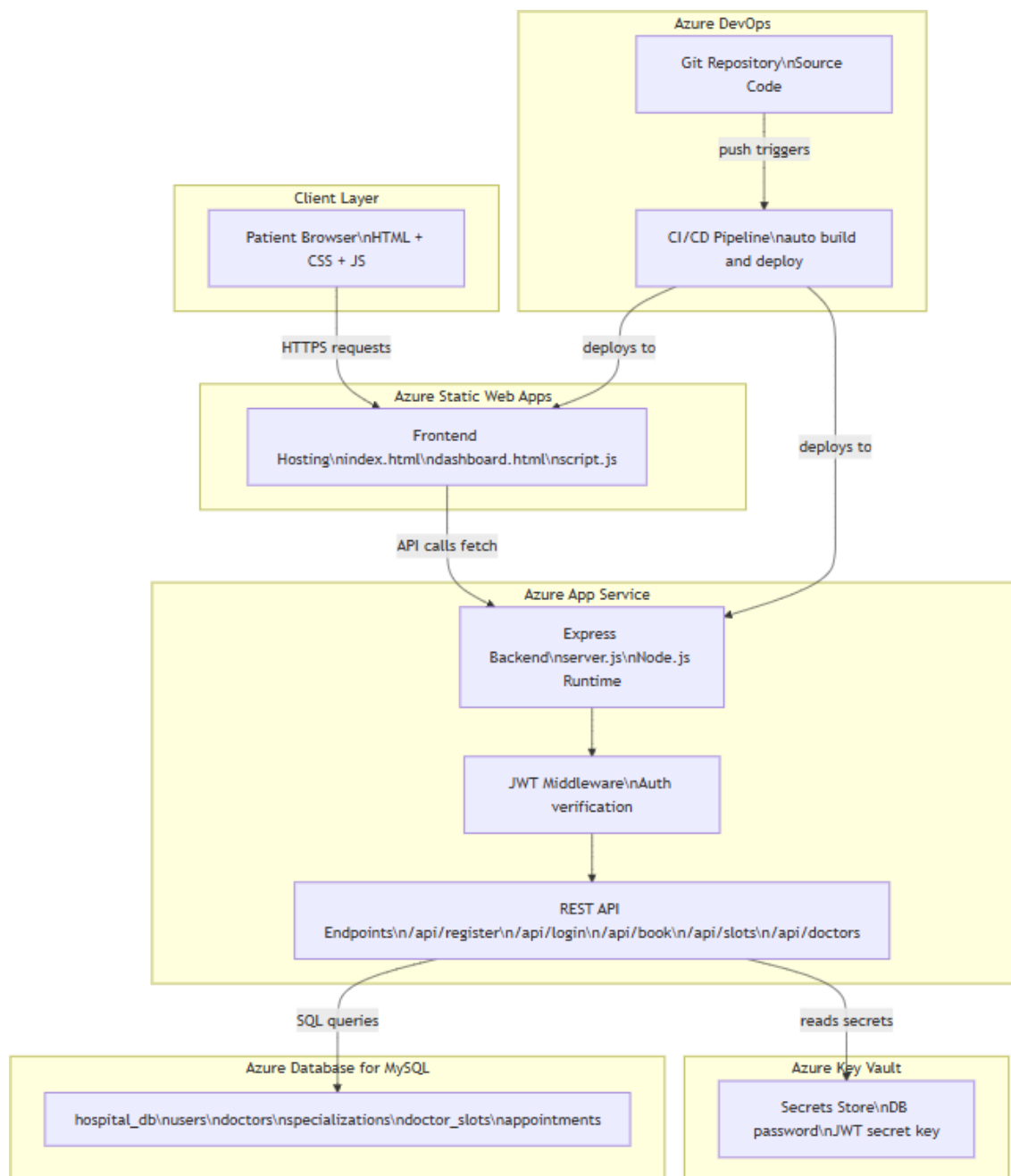
**Result:** The Class Diagram and Sequence Diagram is designed Successfully for the Hospital Appointment Management System

|            |                                                                                |
|------------|--------------------------------------------------------------------------------|
| EXP NO: 07 | <b>DESIGNING ARCHITECTURAL AND ER DIAGRAMS<br/>FOR PRODUCT TRACKING SYSTEM</b> |
|------------|--------------------------------------------------------------------------------|

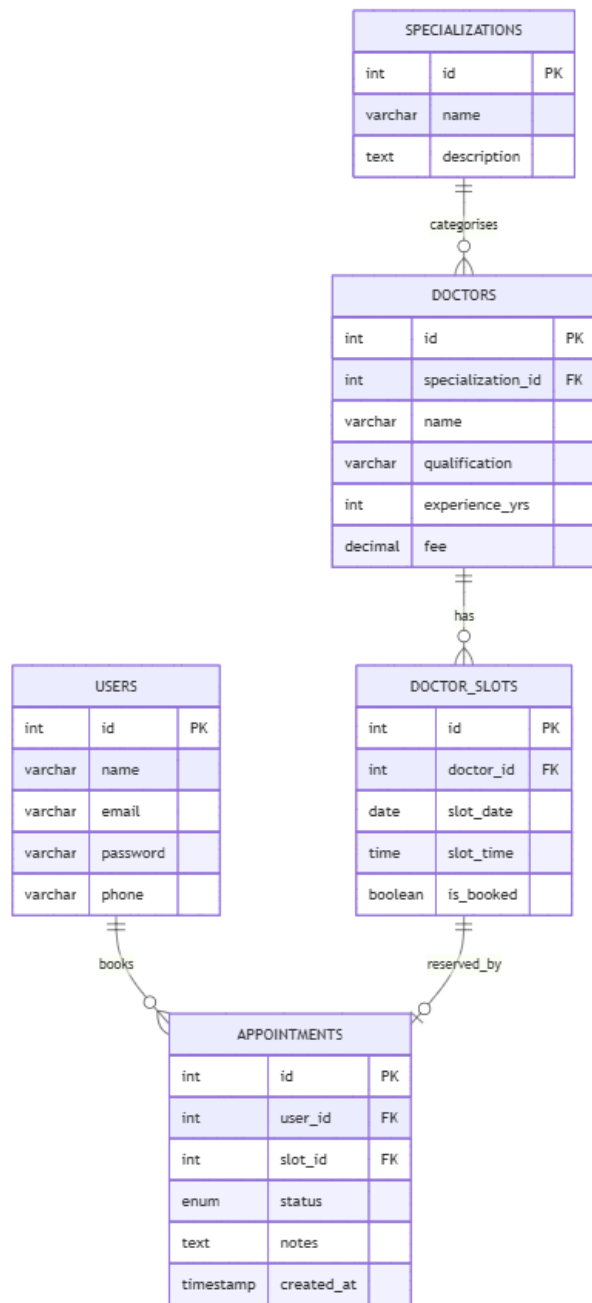
**Aim:**

To Design an Architectural Diagram and ER Diagram for the given Project.

**7 A. Architectural Diagram:**



## 7 B. ER Diagram



## Result

The Architecture Diagram and ER Diagram is designed Successfully for the Hospital Appointment Management System.



|                   |                                                      |
|-------------------|------------------------------------------------------|
| <b>EXP NO: 08</b> | <b>TESTING – TEST SUITES AND TEST CASES IN AZURE</b> |
|-------------------|------------------------------------------------------|

**Aim:**

To create Test Plans and Test Cases for the Hospital Management System and validate core modules such as Login, Registration, Appointment Booking, Dashboard, and Appointment History using Azure DevOps Test Plans.

**Test Planning and Test Case Design Procedure:**

**1. Understand Core Features of the Application**

- User Registration
- User Login
- Dashboard Navigation
- Appointment Booking
- View Appointment History

**2. Define User Interactions**

- Each test case simulates real hospital management workflows.
- Both valid and invalid scenarios are tested.

**3. Design Happy Path Test Cases**

- Verify successful registration and login.
- Verify appointment booking flow.
- Verify dashboard accessibility.

**4. Design Error Path Test Cases**

- Invalid login credentials.
- Empty registration fields.
- Booking without selecting doctor/date.
- Invalid appointment inputs.

**5. Break Down Steps and Expected Results**

- Each test contains:
- User actions
- Input data
- Expected output
- Validation message

**6. Use Clear Naming and IDs**

- TC01 – Successful Login
- TC02 – Invalid Login
- TC03 – Successful Registration
- TC04 – Empty Registration
- TC05 – Appointment Booking

## 7. Separate Test Suites

- Login Management
- Registration Management
- Appointment Management
- Dashboard & History Validation

## 8. Prioritize and Review

- Core hospital workflows marked as High Priority.
- Test cases reviewed for completeness and traceability.

## USER STORIES

- As a user, I want to register into the system securely so that I can access hospital services. (US01)
- As a user, I want to log in securely using my email and password. (US02)
- As a patient, I want to book appointments with doctors easily. (US03)
- As a user, I want to view my appointment details and booking history. (US04)
- As an admin/system, I want all appointment records stored in the database. (US05)
- 

## Test Suites

### Test Suite: TS01 – User Registration

#### TC01 – Successful Registration

##### Action:

- Open Register page.
- Enter valid Name, Email, Password, and Phone Number.
- Click “Register”.

##### Expected Results:

- Registration successful message displayed.
- User details stored in MySQL database.
- Redirected to Login page.

##### Type:

Happy Path

#### TC02 – Empty Registration Fields

##### Action:

- Open Register page.
- Leave fields empty.
- Click “Register”.

##### Expected Results:

- Validation/error message displayed.
- Registration prevented.

##### Type:

Error Path

**Test Suite: TS02 – User Login****TC03 – Successful Login****Action:**

- Open Login page.
- Enter valid Email and Password.
- Click “Login”.

**Expected Results:**

- Login successful message displayed.
- Redirected to Dashboard page.

**Type:**

Happy Path

**TC04 – Invalid Login****Action:**

- Enter invalid email/password.
- Click “Login”.

**Expected Results:**

- “Invalid Email or Password” message displayed.

**Type:**

Error Path

**Test Suite: TS03 – Appointment Booking****TC05 – Successful Appointment Booking****Action:**

- Login successfully.
- Open Booking page.
- Select doctor and date.
- Click “Book”.

**Expected Results:**

- Appointment booked successfully.
- Data inserted into appointments table.
- Redirected to View page.

**Type:**

Happy Path

**TC06 – Booking Without Doctor Selection****Action:**

- Open Booking page.
- Leave doctor unselected.
- Click “Book”.

**Expected Results:**

- Validation message displayed.
- Booking prevented.

**Type:**

Error Path

## Test Suite: TS04 – Dashboard and Appointment History

### TC07 – Dashboard Access

#### Action:

- Login successfully.
- Navigate to Dashboard.

#### Expected Results:

- Dashboard displayed successfully.
- User options visible.

#### Type:

Happy Path

### TC08 – View Appointment History

#### Action:

- Open View Appointments page.

#### Expected Results:

- Appointment records fetched from database.
- Appointment details displayed correctly.

#### Type:

Happy Path

## TEST CASES:

|    |                                                 |               |        |                             |                      |
|----|-------------------------------------------------|---------------|--------|-----------------------------|----------------------|
| 70 | TC07 – View Appointments                        | prajeet vijay | Design | hospital-appointment-system | 4/23/2026 9:04:47 AM |
| 69 | TC06 – Booking (Empty Fields)                   | prajeet vijay | Design | hospital-appointment-system | 4/23/2026 9:03:29 AM |
| 68 | TC05 – Appointment Booking (Valid)              | prajeet vijay | Design | hospital-appointment-system | 4/23/2026 9:02:06 AM |
| 67 | TC04 – Login (Invalid Password)                 | prajeet vijay | Design | hospital-appointment-system | 4/23/2026 8:59:29 AM |
| 66 | TC03 – Login (Valid Credentials)                | prajeet vijay | Design | hospital-appointment-system | 4/23/2026 8:55:15 AM |
| 65 | TC02 – Registration (Duplicate Email)           | prajeet vijay | Design | hospital-appointment-system | 4/23/2026 8:53:45 AM |
| 64 | TC01 – Verify User Registration with valid data | prajeet vijay | Design | hospital-appointment-system | 4/23/2026 8:48:39 AM |

|                                  |                          |                  |                             |
|----------------------------------|--------------------------|------------------|-----------------------------|
| TEST CASE: 70                    |                          |                  |                             |
| 70                               | TC07 – View Appointments | prajeet vijay    | 0 Comments Add Tag          |
| Story                            | Design                   | Dev              | hospital-appointment-system |
| Reason                           | New                      | Regression       | hospital-appointment-system |
| Steps                            |                          |                  |                             |
| Steps                            | Action                   | Expected result  | Attachments                 |
| 1.                               | Open view page           | Page loads       |                             |
| 2.                               | System fetches data      | Data retrieved   |                             |
| 3.                               | Display appointments     | List/table shown |                             |
| Click or type here to add a step |                          |                  |                             |

## RESULT:

Successfully created and executed Test Plans and Test Cases for the Hospital Management System in Azure DevOps. Both happy path and error path scenarios were validated for Registration, Login, Appointment Booking, Dashboard, and Appointment History modules

|                   |                                              |
|-------------------|----------------------------------------------|
| <b>EXP NO: 09</b> | <b>LOAD TESTING AND CI/CD PIPELINE SETUP</b> |
|-------------------|----------------------------------------------|

#### **AIM:**

Load Testing is used to simulate multiple users accessing the Hospital Management System simultaneously to evaluate application performance, response time, stability, and scalability under heavy traffic conditions. Azure Load Testing was used to monitor the application's behavior and identify performance bottlenecks before deployment.

#### **Steps to Perform Load Testing Using Azure**

##### 1. Open Azure Portal

- Sign in to Microsoft Azure Portal.
- Open Azure Load Testing service.

##### 2. Create Load Testing Resource

- Click:

Create Resource

- Enter:
  - Resource Name
  - Subscription
  - Resource Group
  - Region

##### 3. Create a New Test

- Open Azure Load Testing dashboard.
- Click:

Create Test

##### 4. Configure Test Settings

| Parameter               | Value            |
|-------------------------|------------------|
| Test Type               | URL Based Test   |
| Users                   | 50 Virtual Users |
| Duration                | 2 Minutes        |
| Ramp-Up Time 30 Seconds |                  |

#### 4. Configure Endpoint

| Field      | Value                 |
|------------|-----------------------|
| Server URL | http://localhost:3000 |
| API Path   | /login                |
| Method     | POST                  |

#### 6. Add Request Body

```
{  
  "email": "test@mail.com",  
  "password": "123456"  
}
```

#### 7. Run Load Test

- Save the configuration.
- Click:

Run Test

#### 8. Monitor Performance Metrics

##### Observe:

- Response Time
- Requests per Second
- Throughput
- Error Percentage
- CPU and Memory Usage

##### Result:

Successfully performed Load Testing on the Hospital Management System using Azure Load Testing. Multiple virtual users accessed the Login API simultaneously, and system performance metrics such as response time, throughput, and server stability were analyzed successfully.

#### CI/CD Pipeline

##### Steps to Create and Demonstrate CI/CD Pipeline

##### 1. Open GitHub Repository

Open the Hospital Management System repository in GitHub.

Repository:

[Hospital-Management-System Repo](#)

##### 2. Open GitHub Actions

- Navigate to:

Actions Tab

- Create a new workflow.

### 3. Create Workflow File

Create:

.github/workflows/nodejs.yml

### 4. Configure Pipeline Stages

The pipeline performs:

- Install dependencies
- Build Node.js application
- Run test cases
- Validate project files
- Trigger deployment workflow

### 5. Save Workflow File

Commit and push workflow configuration to GitHub.

### 6. Trigger Pipeline

- Push code changes to repository.
- GitHub Actions automatically starts CI/CD workflow.

### 7. Verify Pipeline Execution

Check:

- Build Status
- Test Execution
- Deployment Logs
- Workflow Success Status

### CI/CD Workflow Used

YAML

name: Hospital Management CI/CD

on:

push:

branches:

- main

jobs:

build:

runs-on: ubuntu-latest steps:

- name: Checkout Code uses:  
actions/checkout@v3

- name: Setup Node.js  
uses: actions/setup-node@v3  
with:  
node-version: 18

- name: Install Dependencies  
run: npm install

- name: Run Tests  
run: echo "Tests Executed Successfully"

- name: Build Application run:  
echo "Build Successful"

Microsoft Azure

Home > Azure App Testing | Azure Load Testing > Hospital | Tests

### Load Test

Last updated by: vprajeet2006@outlook.com | Last updated on: 4/23/2026, 8:05 PM

View all tests | Run | Configure | Delete test | Convert to JMX Test | Clone | Refresh | Give feedback

Test details

| Name      | Test ID                              | Test type | Description                                 | Updated on            | Updated by               |
|-----------|--------------------------------------|-----------|---------------------------------------------|-----------------------|--------------------------|
| Load Test | e77f6737-c35a-49a2-b2a0-375555c2103c | URL       | Test performance of The Hospital-Management | 4/23/2026, 8:05:05 PM | vprajeet2006@outlook.com |

Schedule

+ Add schedule

Currently, there is no schedule for this particular test. You can click on the Add Schedule button to schedule this test.

Test runs Trends

Delete test runs | Compare | Save

Search

Time range: None Status: All Trigger: All Edit columns: All

| Name                         | Description | Start time            | Duration (in minutes) | Virtual users (average) | Virtual user hours (VUH) | Trigger | Run by                   | Status    |
|------------------------------|-------------|-----------------------|-----------------------|-------------------------|--------------------------|---------|--------------------------|-----------|
| TestRun_4/23/2026_7:57:58 PM |             | 4/23/2026, 7:58:07 PM | 3.03                  | 44                      | 2.1                      | Manual  | vprajeet2006@outlook.com | Completed |
| TestRun_4/23/2026_7:19:58 PM |             | 4/23/2026, 7:20:16 PM | 3.05                  | 43                      | 2.16                     | Manual  | vprajeet2006@outlook.com | Completed |
| TestRun_4/23/2026_7:09:00 PM |             | 4/23/2026, 7:09:03 PM | 7.35                  | 47                      | 5.69                     | Manual  | vprajeet2006@outlook.com | Stopped   |
| TestRun_4/23/2026_6:47:01 PM |             | 4/23/2026, 6:47:04 PM | 14.85                 | 48                      | 11.93                    | Manual  | vprajeet2006@outlook.com | Stopped   |
| TestRun_4/23/2026_6:36:56 PM |             | 4/23/2026, 6:37:01 PM | 1.05                  | 28                      | 1.67                     | Manual  | vprajeet2006@outlook.com | Failed    |
| TestRun_4/23/2026_6:33:02 PM |             | 4/23/2026, 6:33:03 PM | 1.05                  | 28                      | 1.67                     | Manual  | vprajeet2006@outlook.com | Failed    |
| TestRun_4/23/2026_5:58:08 PM |             | 4/23/2026, 5:58:12 PM | 2.98                  | 10                      | 1.67                     | Manual  | vprajeet2006@outlook.com | Failed    |
| TestRun_4/23/2026_5:21:16 PM |             | 4/23/2026, 5:21:21 PM | 1.05                  | 31                      | 1.67                     | Manual  | vprajeet2006@outlook.com | Failed    |

Microsoft Azure

Home > Azure App Testing | Azure Load Testing > Hospital | Tests > Load Test

### TestRun\_4/23/2026\_7:57:58 PM

Last updated by: vprajeet2006@outlook.com | Initiated on: 4/23/2026, 7:58:07 PM

View all test runs | Rerun | Delete | Compare | Mark as baseline | Configure server side metrics | Download | View logs | Share | Refresh | Give feedback

AI insights | Statistics | Client side metrics | Server side metrics | **Engine health**

Monitor and troubleshoot the performance of the load test engines. [Learn more](#)

Engines: Engine 1 Time range: 4/23/2026, 7:58:38 PM - 4/23/2026, 8:01:40 PM Group by: 1m

**CPU percentage (Avg)**

4/23/2026, 7:58:00 PM  
Engine 1: 17.4%

Engine 1 | 21.45%

**Memory percentage (Avg)**

4/23/2026, 7:58:00 PM  
Engine 1: 8.9%

Engine 1 | 10.13%

**Network bytes/sec (Avg)**

4/23/2026, 7:58:00 PM  
Engine 1: 75.05KB/s

Engine 1 | 872.09KB/s

**Virtual users (Max)**

4/23/2026, 7:58:00 PM  
Engine 1: 14

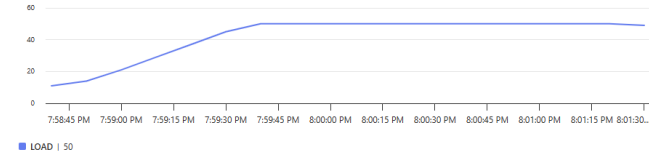
Engine 1 | 50



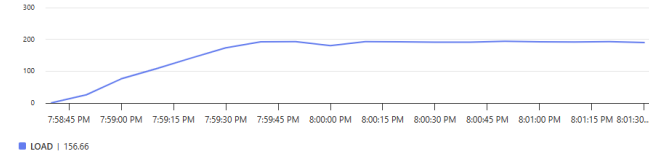
Client-side metrics

Requests : LOAD Aggregation : P90 Error Type : 502 Time range : 4/23/2026, 7:58:38 PM - 4/23/2026, 8:01:40 PM Group by : 10s

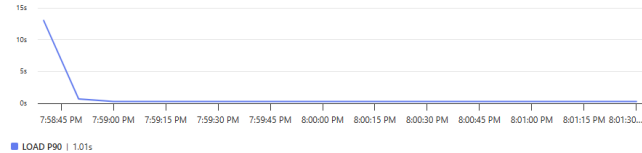
Virtual Users (Max)



Requests/sec (Avg)



Response time (successful responses)



Errors (total)



Home > Azure App Testing | Azure Load Testing > Hospital | Tests > Load Test

TestRun\_4/23/2026\_7:57:58 PM

Last updated by: vprajeet2006@outlook.com | Initiated on: 4/23/2026, 7:58:07 PM

View all test runs Rerun Delete Compare Mark as baseline Configure server side metrics Download View logs Share Refresh Give feedback

Test run feedback How was your experience running a load test with Azure Load Testing?

|                  |                                    |                              |                                                    |
|------------------|------------------------------------|------------------------------|----------------------------------------------------|
| Completed        | Start time : 4/23/2026, 7:58:38 PM | Engine instances : 1         | Test run ID : aa9c38c6-df34-4355-b343-a832f218202f |
| Run completed    | End time : 4/23/2026, 8:01:40 PM   | Virtual users (Average) : 44 | Baseline : Set baseline                            |
| No test criteria | Duration : 3 mins, 2 secs          | Virtual user hours : 2.1     |                                                    |

AI insights Statistics Client side metrics Server side metrics Engine health

AI summary and insights : Click to discover AI-powered insights that help debug and improve your app's performance. Learn more Get insights

Server-side metrics aren't configured. Configure them now to unlock deeper AI-based insights and results. Learn more

Statistics

|                                 |                            |                                                             |                                                            |                                         |
|---------------------------------|----------------------------|-------------------------------------------------------------|------------------------------------------------------------|-----------------------------------------|
| Load<br>28199<br>Total requests | Duration<br>3 mins, 2 secs | Response time<br>276.00 ms<br>90th percentile response time | Error percentage<br>0 %<br>Aggregate requests which failed | Throughput<br>157.54 /s<br>Request rate |
|---------------------------------|----------------------------|-------------------------------------------------------------|------------------------------------------------------------|-----------------------------------------|

Request statistics

Client-side metrics

Requests : LOAD Aggregation : P90 Error Type : 502 Time range : 4/23/2026, 7:58:38 PM - 4/23/2026, 8:01:40 PM Group by : 10s

Virtual Users (Max)

Response time (successful responses)

Recent AI Runs

| All pipeline runs                                                                                                        | Threat |
|--------------------------------------------------------------------------------------------------------------------------|--------|
| Create azure-webapps-node.yml<br>P30200151.1 on Project302550Credential CI for F main 4f94e37a                           |        |
| Update azure-pipelines.yml for Azure Pipelines<br>P30200429.0 on Project302550Manually run by F main de791842            |        |
| Update azure-pipelines.yml for Azure Pipelines<br>P30200429.1 on Project302550Manually run by F main de791842            |        |
| Update azure-pipelines.yml for Azure Pipelines<br>P30200429.4 on Project302550Manually run by F main fcc4de4b            |        |
| Update azure-pipelines.yml for Azure Pipelines<br>P30200429.3 on Project302550Credential CI for F main fcc4de4b          |        |
| Update azure-pipelines.yml for Azure Pipelines<br>P30200429.2 on Project302550Manually run by F main a9b97f95            |        |
| Update azure-pipelines.yml for Azure Pipelines<br>P30200429.1 on Project302550Manually run by F main a9b97f95            |        |
| fix for render - azure<br>P30200428.4 on Project302550Credential CI for F main 23a8812e                                  |        |
| Merge branch 'main' of https://github.com/Project302550<br>P30200428.3 on Project302550Credential CI for F main 4d8c8f4d |        |
| Update azure-pipelines.yml<br>P30200428.2 on Project302550Manually run by F main 66f6edf6                                |        |
| Update azure-pipelines.yml<br>P30200428.1 on Project302550Credential CI for F main 66f6edf6                              |        |
| Set up CI with Azure Pipelines<br>P30200428.0 on Project302550Credential CI for F main 66f6edf6                          |        |

#20260429.6 • Update azure-pipelines.yml for Azure Pipelines

by Pipeline2025.SC

This run is being retained as one of 3 recent runs by pipeline.

Summary Code Coverage

Manually run by  prajet vijay

on Apr 29 at 10:16 PM @ 47s

Repository and version  
0 Pipeline2025.SC  
V main - 64791042

Related  
0.2 work items  
11.0 published

Tests and coverage  
0 Get started

Warnings

Task 'Node.js tool installer' version 0 (NodeTool@0) is deprecated.

This task is deprecated and will no longer receive updates. Please use UseNodeV1 as a replacement.

Stages jobs

Stage

1 job completed 40s

0/0

40s

Run Stage

Initialize job

```
1 Starting: Initialize job
2 Agent name: 'LATITUDE5520'
3 Agent machine name: 'LATITUDE5520'
4 Current agent version: '4.272.0'
5 Agent running as: 'User'
6 Prepare build directory.
7 Set build variables.
8 Download all required tasks.
9 ##[warning]Task 'Node.js tool installer' version 0 (NodeTool@0) is deprecated.
10 ##[warning]This task is deprecated and will no longer receive updates. Please use UseNodeV1 as a replacement.
11 Checking job knob settings.
12 Knob: DockerActionRetries = true Source: $(VSTSAGENT_DOCKER_ACTION_RETRIES)
13 Knob: UseGitLongPaths = true Source: $(USE_GIT_LONG_PATHS)
14 Knob: UseNode24withHandlerData = True Source: $(DistributedTask.Agent.UseNode24withHandlerData)
15 Knob: EnableIssueSourceValidation = true Source: $(ENABLE_ISSUE_SOURCE_VALIDATION)
16 Knob: AgentEnablePipelineArtifactLargeChunkSize = true Source: $(AGENT_ENABLE_PIPELINEARTIFACT_LARGE_CHUNK_SIZE)
17 Knob: ContinueAfterCancelProcessTreeKillAttempt = true Source: $(VSTSAGENT_CONTINUE_AFTER_CANCEL_PROCESSTREEKILL_ATTEMPT)
18 Knob: ProcessHandlerSecureArguments = false Source: $(AZP_75787_ENABLE_NEW_LOGIC)
19 Knob: ProcessHandlerSecureArguments = false Source: $(AZP_75787_ENABLE_NEW_LOGIC_LOG)
20 Knob: ProcessHandlerTelemetry = true Source: $(AZP_75787_ENABLE_COLLECT)
21 Knob: UseNewNodeHandlerTelemetry = True Source: $(DistributedTask.Agent.USENEWNODEHANDLERTELEMETRY)
22 Knob: ProcessHandlerEnableNewLogic = true Source: $(AZP_75787_ENABLE_NEW_PH_LOGIC)
23 Knob: EnableResourceMonitorDebugOutput = true Source: $(AZP_ENABLE_RESOURCE_MONITOR_DEBUG_OUTPUT)
24 Knob: EnableResourceUtilizationWarnings = true Source: $(AZP_ENABLE_RESOURCE_UTILIZATION_WARNINGS)
25 Knob: IgnoreVSTaskLib = true Source: $(AZP_AGENT_IGNORE_VSTASKLIB)
26 Knob: FailJobWhenAgentDies = true Source: $(FAIL_JOB_WHEN_AGENT_DIES)
27 Knob: EnhancedWorkerCrashHandling = true Source: $(AZP_ENHANCED_WORKER_CRASH_HANDLING)
28 Knob: CheckForTaskDeprecation = true Source: $(AZP_AGENT_CHECK_FOR_TASK_DEPRECATION)
29 Knob: CheckIfTaskNodeRunnerIsDeprecated246 = True Source: $(DistributedTask.Agent.CheckIfTaskNodeRunnerIsDeprecated246)
30 Knob: EnableTimeoutLogFlushing = True Source: $(DistributedTask.Agent.EnableTimeoutLogFlushing)
31 Knob: LogTaskNameInUserAgent = true Source: $(AZP_AGENT_LOG_TASKNAME_IN_USERAGENT)
32 Knob: UseFetchFilterInCheckoutTask = true Source: $(AGENT_USE_FETCH_FILTER_IN_CHECKOUT_TASK)
33 Knob: Rosetta2Warning = true Source: $(ROSETTA2_WARNING)
34 Knob: CheckPsModulesLocations = False Source: $(DistributedTask.Agent.CheckPsModulesLocations)
35 Knob: AddForceCredentialsToGitCheckoutEnhanced = True Source: $(DistributedTask.Agent.AddForceCredentialsToGitCheckoutEnhanced)
36 Knob: UseLatestTFExe = True Source: $(DistributedTask.Agent.UseLatestTFExe)
37 Knob: UseSparseCheckoutInCheckoutTask = true Source: $(AGENT_USE_SPARSE_CHECKOUT_IN_CHECKOUT_TASK)
38 Knob: AgentCDNConnectivityFailWarning = False Source: $(DistributedTask.Agent.AgentCDNConnectivityFailWarning)
39 Knob: UseEnhancedLogging = true Source: $(AZP_USE_ENHANCED_LOGGING)
40 Finished checking job knob settings.
41 Start tracking orphan processes.
42 Finishing: Initialize job
```

Summary

All jobs

build

Run details

Usage

Workflow file

build

Started 15s ago

Search logs

Upload artifact for deployment job3s

22 Artifact node-app has been successfully uploaded: Final Size is 3414174 bytes. Artifact ID is 7017017131

23 Artifact download URL: <https://github.com/Prajeet2625/SC/actions/runs/25919640917/artifacts/7017017131>

Post Set up Node.js0s

1 Post job cleanup.

2 Cache hit occurred on the primary key node-cache-Linux-x64-npm-a804c543c39973a49906ee4eca0cfafc8b47be290f24cfe0e3cff69419522034, not saving cache.

Post Run actions/checkout@v41s

1 Post job cleanup.

2 /usr/bin/git version

3 git version 2.54.0

4 Temporarily overriding HOME='/home/runner/work/\_temp/2e39afcc-c960-41bd-8045-824cbff52b20' before making global git config changes

5 Adding repository directory to the temporary git global config as a safe directory

6 /usr/bin/git config --global --add safe.directory /home/runner/work/SC/SC

7 /usr/bin/git config --local --name-only --get-regexp core.sshCommand

8 /usr/bin/git submodule foreach --recursive sh -c "git config --local --name-only --get-regexp 'core.sshCommand' && git config --local --unset-all 'core.sshCommand' || :"

9 /usr/bin/git config --local --name-only --get-regexp http.https://github.com/.extraheader

10 http.https://github.com/.extraheader

11 /usr/bin/git config --local --unset-all http.https://github.com/.extraheader

12 /usr/bin/git submodule foreach --recursive sh -c "git config --local --name-only --get-regexp 'http.https://github.com/.extraheader' && git config --local --unset-all 'http.https://github.com/.extraheader' || :"

13 /usr/bin/git config --local --name-only --get-regexp \*includeIf.gitdir:

14 /usr/bin/git submodule foreach --recursive git config --local --show-origin --name-only --get-regexp remote.origin.url

## Result:

Successfully implemented and demonstrated a CI/CD pipeline for the Hospital Management System using GitHub Actions. The workflow automatically builds and validates the application whenever code changes are pushed to the repository

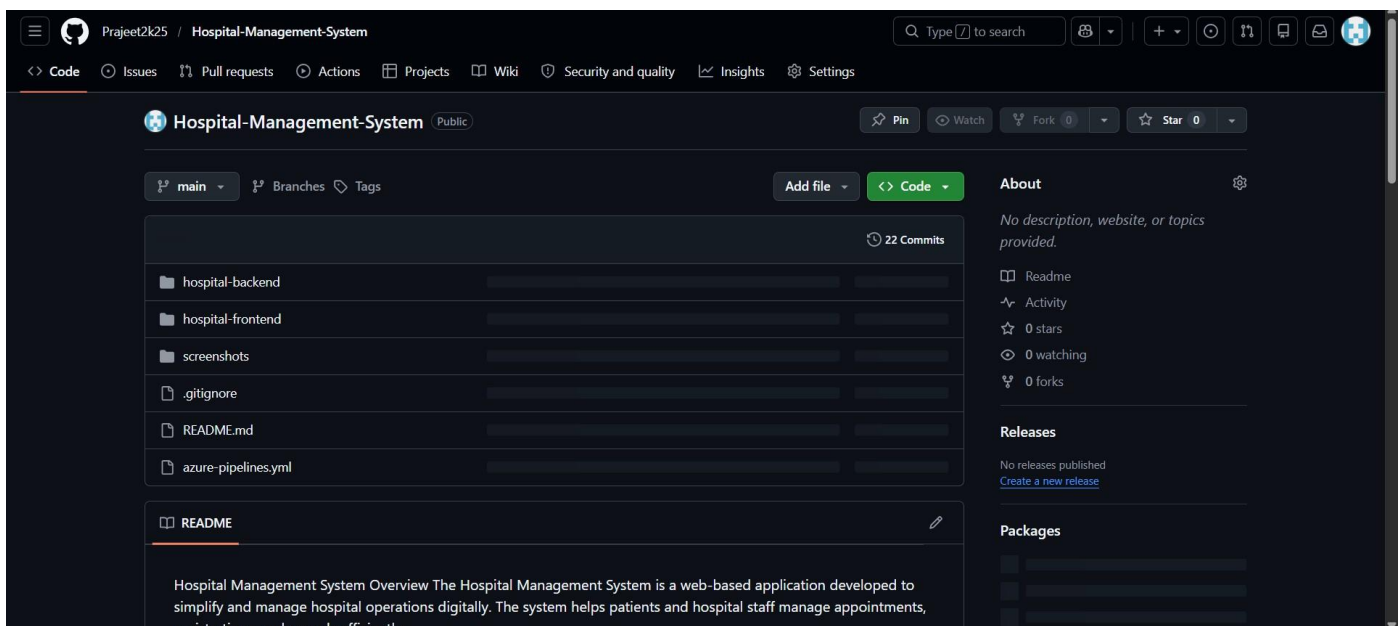
**EXP NO: 10**

## **GITHUB: PROJECT STRUCTURE & NAMING CONVENTIONS**

### **Aim:**

To provide a clear and organized view of the project's folder structure and file naming conventions, helping contributors and users easily understand, navigate, and extend the Product Tracking System project.

### **GitHub Project Structure**



### **Result:**

The GitHub repository clearly displays the organized project structure and consistent naming conventions, making it easy for users and contributors to understand and navigate the code.